# Channel Zero: Breaking and Building Radio Security

## micro:bit

**Dr. Mike Ham**

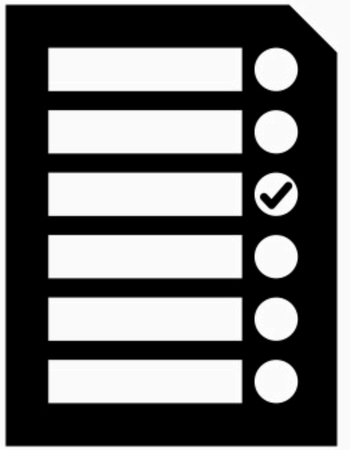
# Goal for Today

- Demonstrate how pre-shared keys control access to a communication channel by programming a micro:bit radio messenger, then exploiting a peer's unencrypted channel to illustrate why channel isolation alone does not constitute secure communication



# Plan for Today

1. Hardware Distribution & Setup
2. “What is this thing?” Orientation
3. First Blink - Sanity check
4. Build: Secret Channel Messenger
    1. Live code together
    2. Flash & test
    3. Eavesdrop
    4. Extension challenge - if there’s time
5. Wrap-up

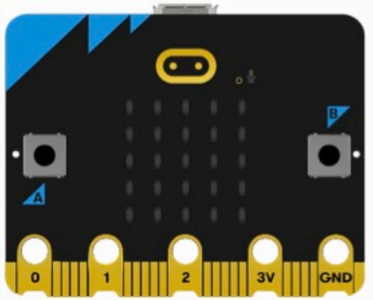


Michael Ham

# 1. Hardware Distribution & Setup

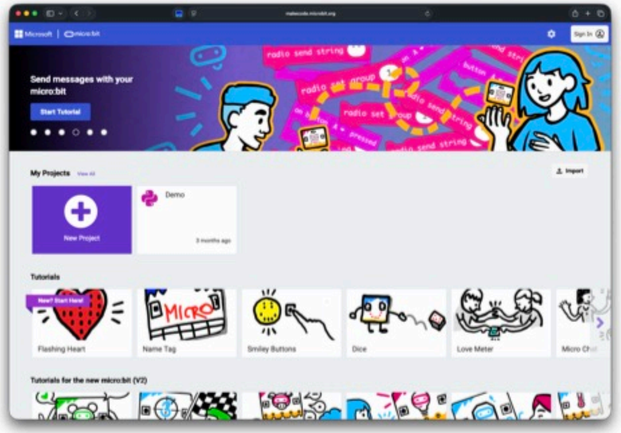
# Hardware Distribution & Setup

1. Everyone needs to have:
    - One micro:bit board
    - One Micro-USB cable
2. Once you have your hardware, plug it into your computer.
    - The microbit will show up as a removable drive called **MICROBIT**
3. Open a web browser (Edge, Chrome, Firefox) and go to **makecode.microbit.org**
4. Wait here! If your neighbors are stuck, help them out.



micro:bit board


Micro USB



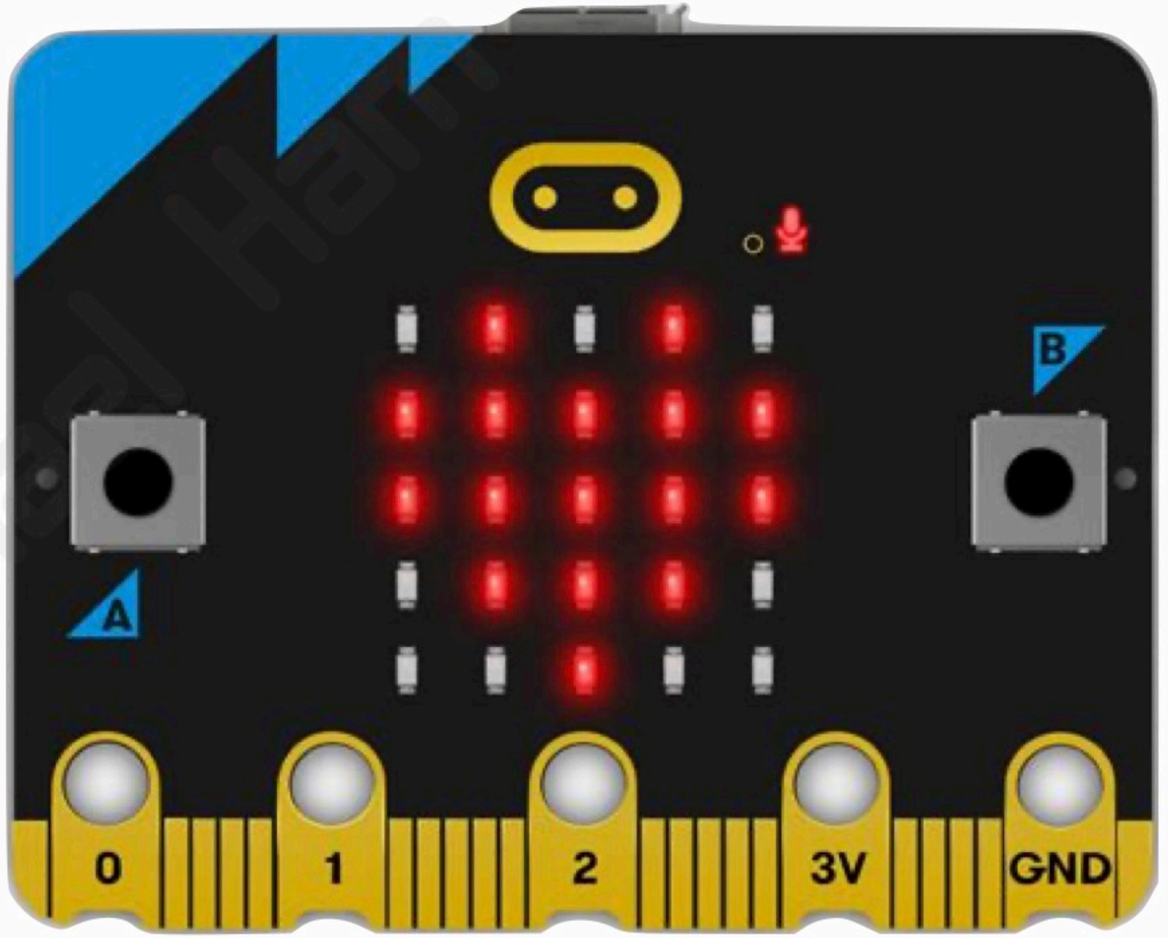
makecode.microbit.org

© Michael Ham

# 2. “What is this thing?” Orientation

# Introducing micro:bit

- A small programmable computer on a circuit board
- Designed for learning programming and electronics
- Contains a microcontroller, sensors, LEDs, radio, and input/ output pins
- Programs run directly on the board's microcontroller
- TL;DR; The micro:bit is a complete embedded system on a tiny PCB



# Main Processor



- Nordic Semiconductor nRF52833 (micro:bit V2)
- Built around an ARM Cortex-M4 CPU
- Earlier boards used nRF51822 with Cortex-M0
- Runs the program
- Conrols sensors, LEDs, and pins
- Manages wireless communications

Michael Ham

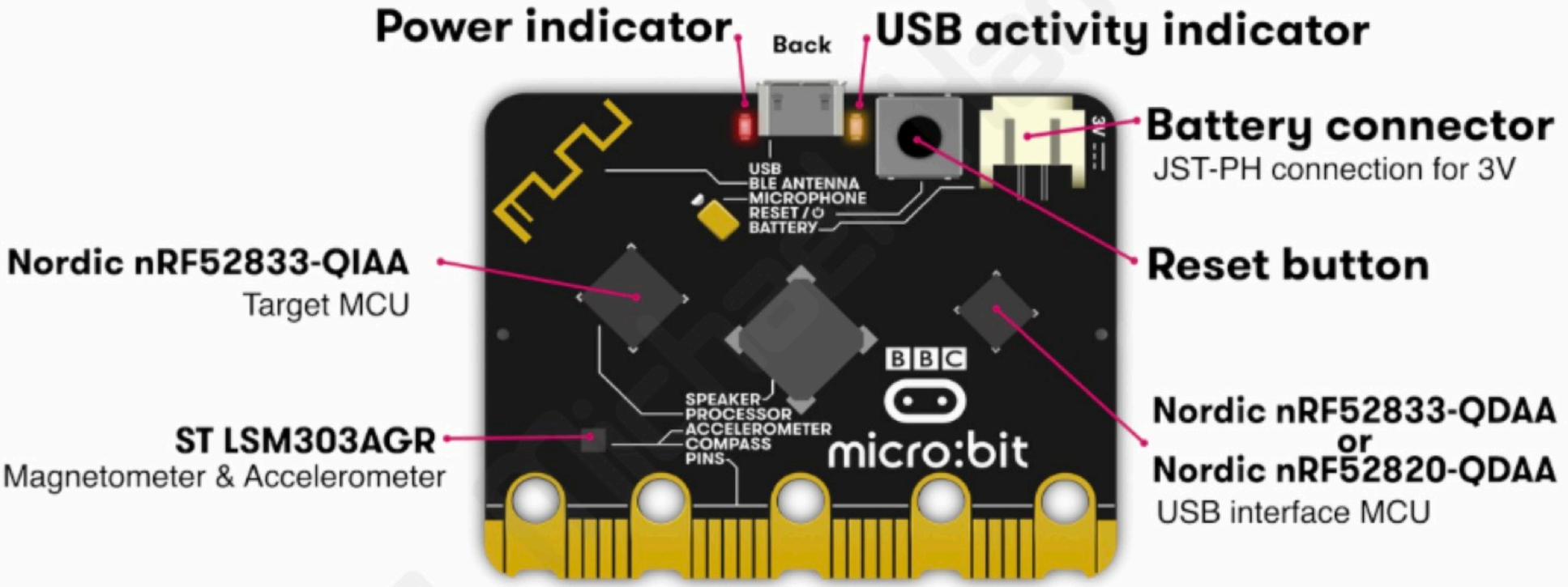
# Secondary Processor



- NXP KL27Z
  - Handles USB communication
  - Allows drag-and-drop programming
  - Provides debugging interface
- The board has two microcontrollers:
  - Target MCU → runs your program
  - Interface MCU → manages USB programming
  - Communication via SWD and UART

Michael Ham

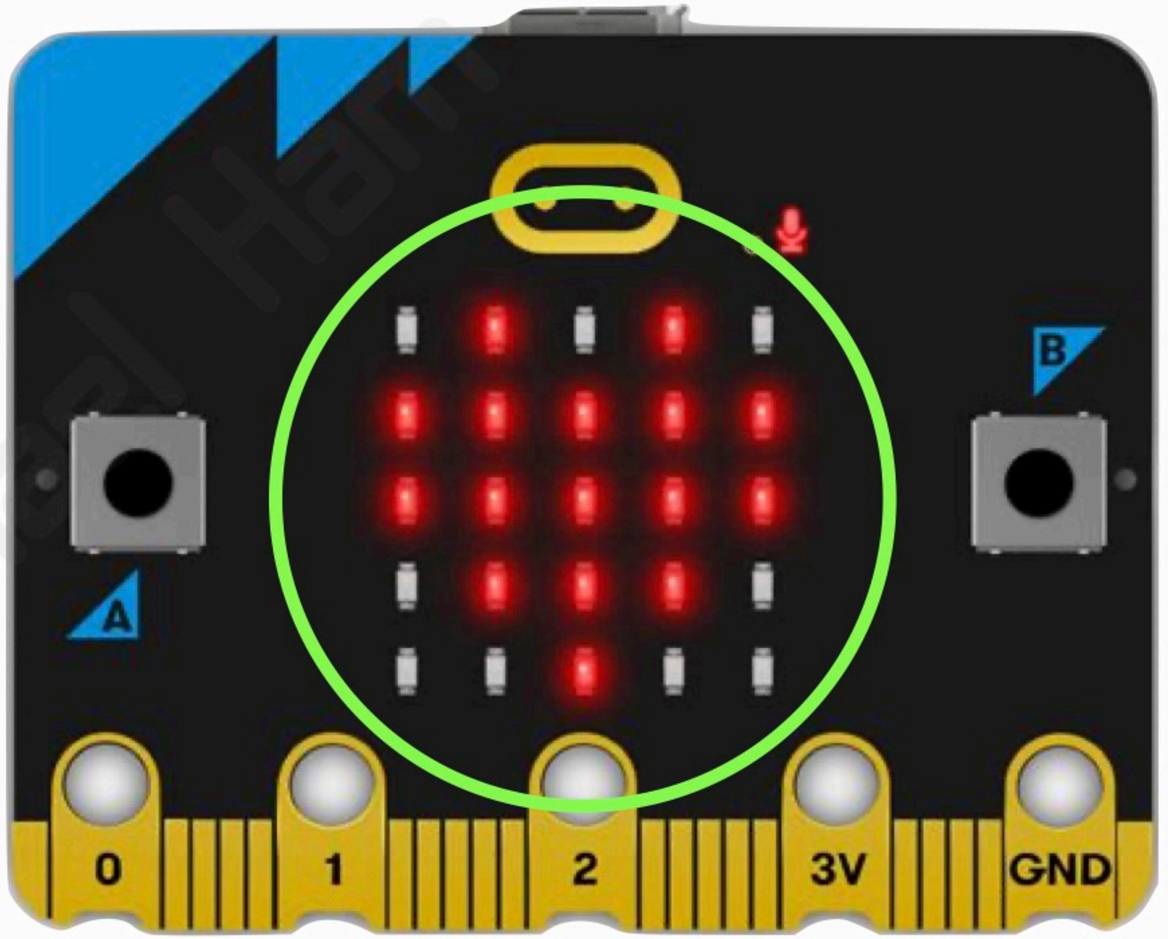
# Chip Layout




Back side of micro:bit

# Display

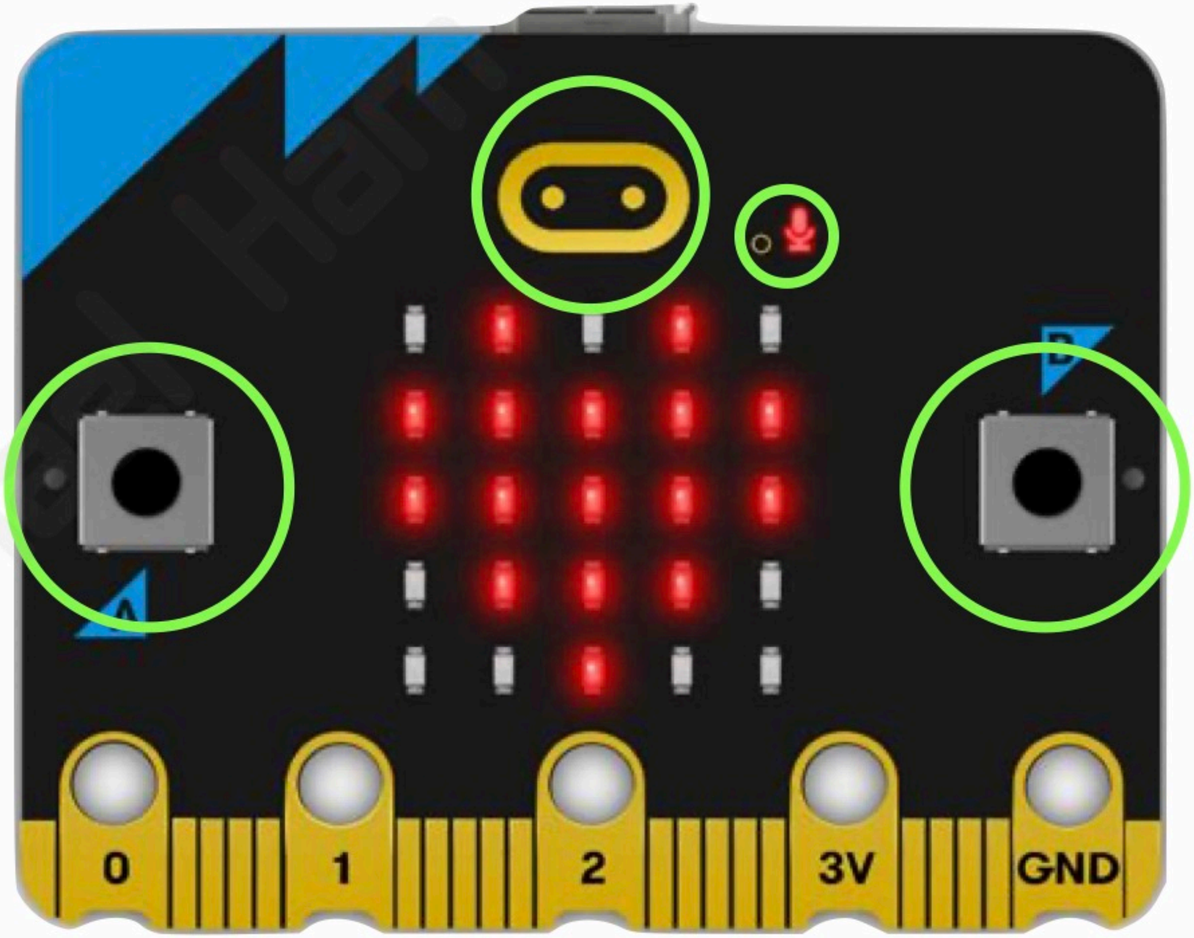
- 25 LEDs
- Adjustable brightness levels
- Can show:
  - icons
  - letters
  - numbers
  - animations
- LEDs can also be used to estimate ambient light levels in software




© Michael Heron

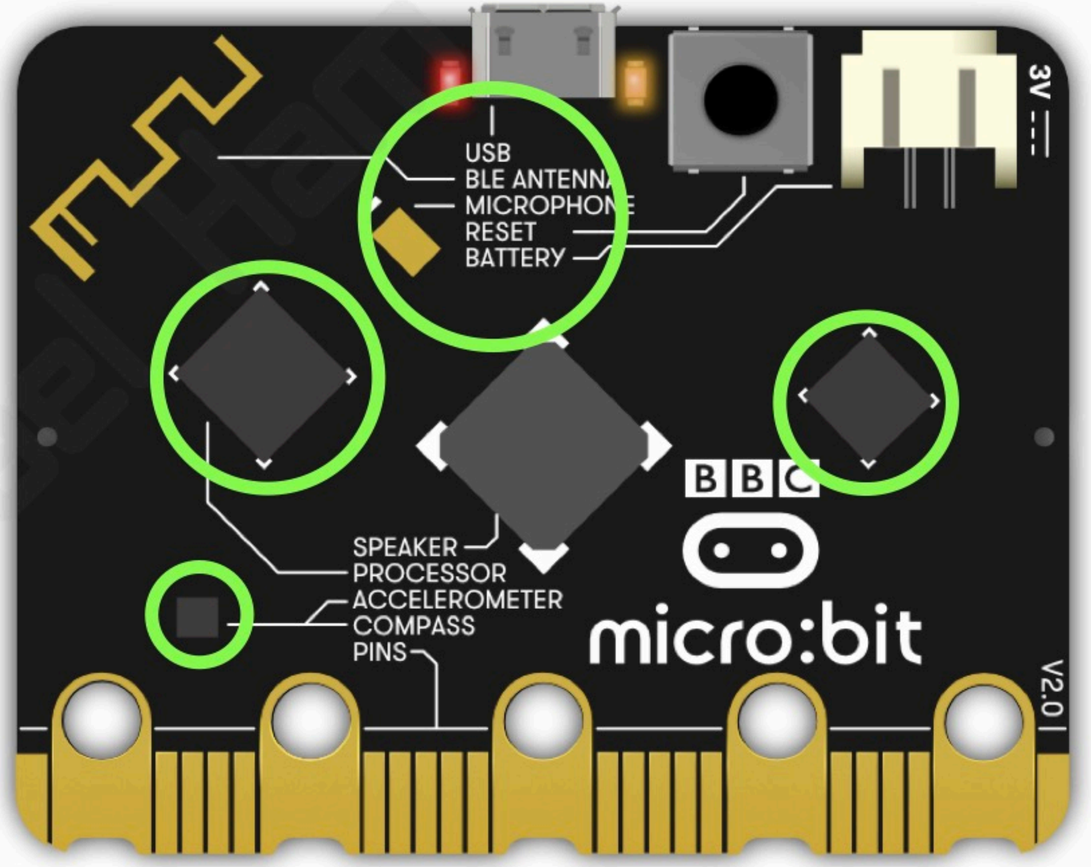
# Buttons and IO

- A and B buttons, along with a touch-sensitive logo
  - Button presses, combinations of buttons
  - Connect to the GPIO pins
- 3-axis accelerometer and magnetometer (compass) - on the back side
  - Tilt, shake, orientation, magnetic direction
- Mirophone, speaker
  - Sound levels, simple audio output



# Wireless

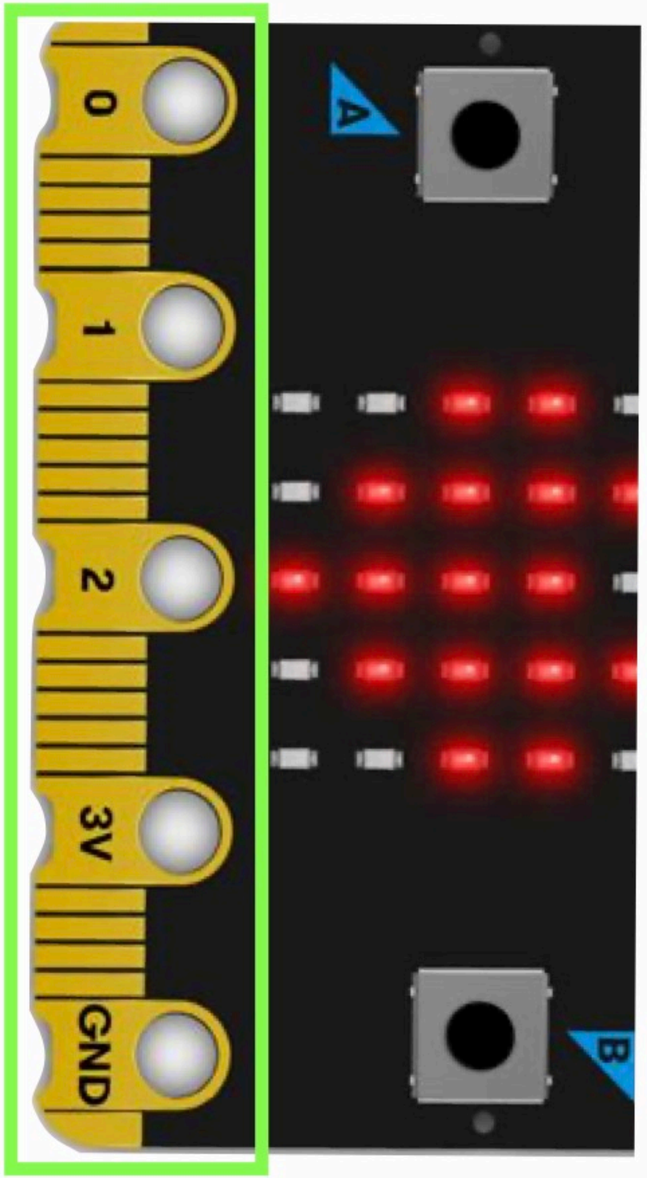
- Bluetooth Low Energy
  - device-to-device radio messaging
  - micro:bit to micro:bit communication
- Smartphone connectivity




Michael Horn

# IO Pins/Edge Connector

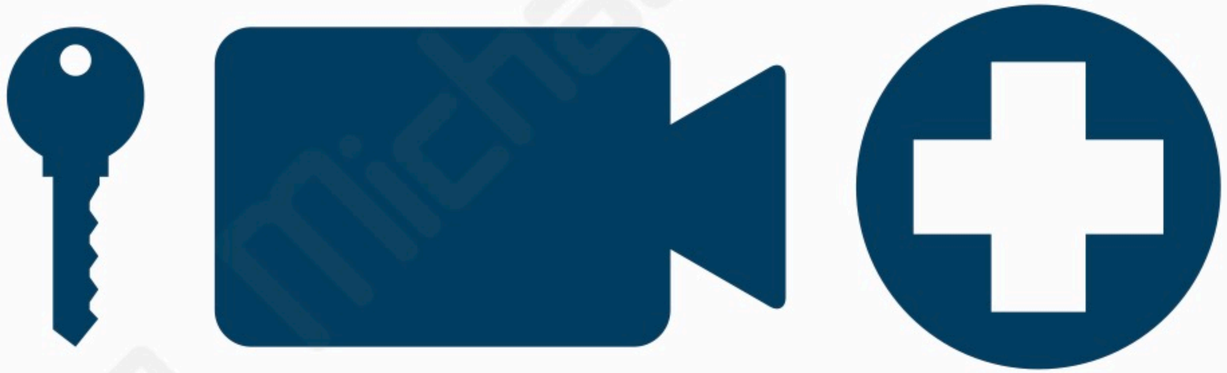
- 25-pin edge connector
  - GPIO pins
  - power (3V)
  - ground
  - analog input
  - digital communication
- Supported Interfaces
  - I2C
  - SPI
  - UART
  - PWM




© Michael Ham

# Takeaways

- It’s an embedded system!
- The same class of hardware is inside smart locks, key fobs, RFID access badges, security cameras, medical devices, etc.



# If you could broadcast a wireless message to every nearby micro:bit with no password, what could go wrong?

**Think-Pair-Share:**
1 minute to write an answer
1 minutes to discuss with your partner
2 mins to come up with an answer as a group

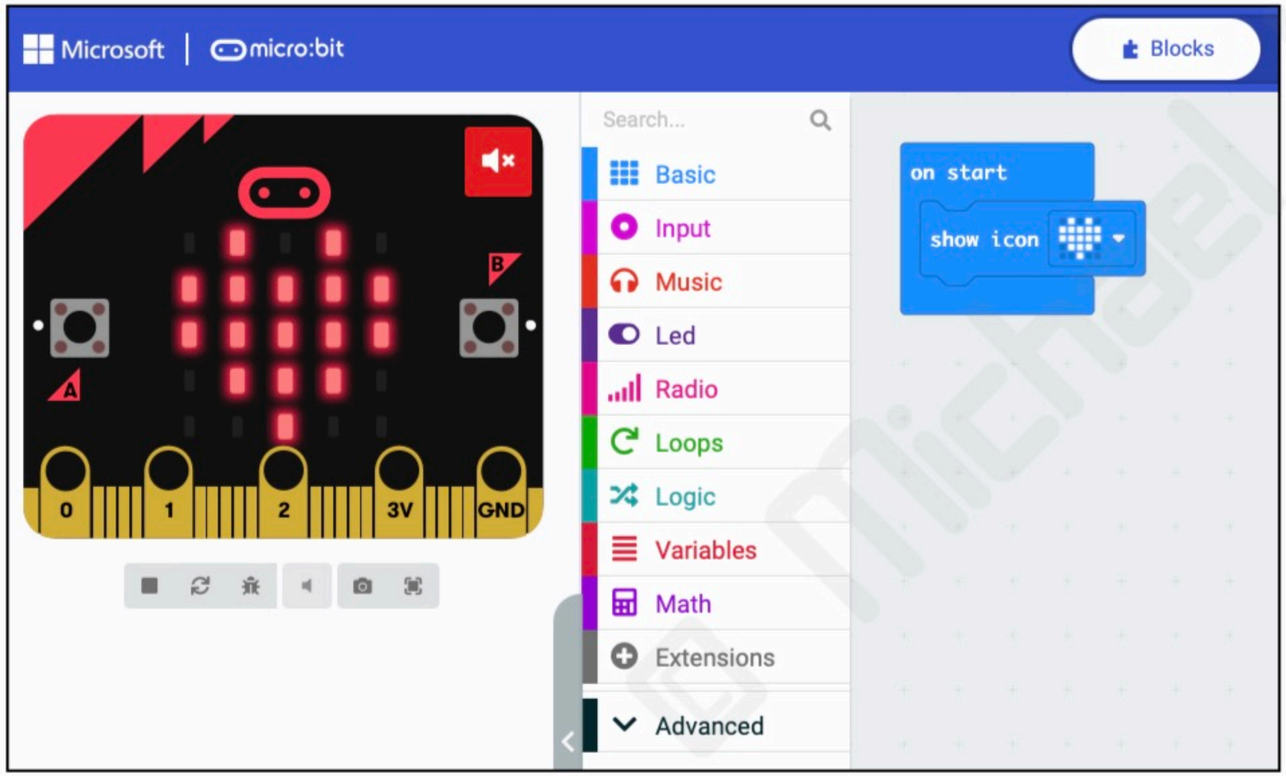
# We’re building a wireless messenger.

**Using a shared secret number to stay on a private channel.**

# 3. First Blink - Sanity Check

# First Blink - Sanity Check

- Goal: confirm our flash and deploy pipeline works before we get to a more complicated project

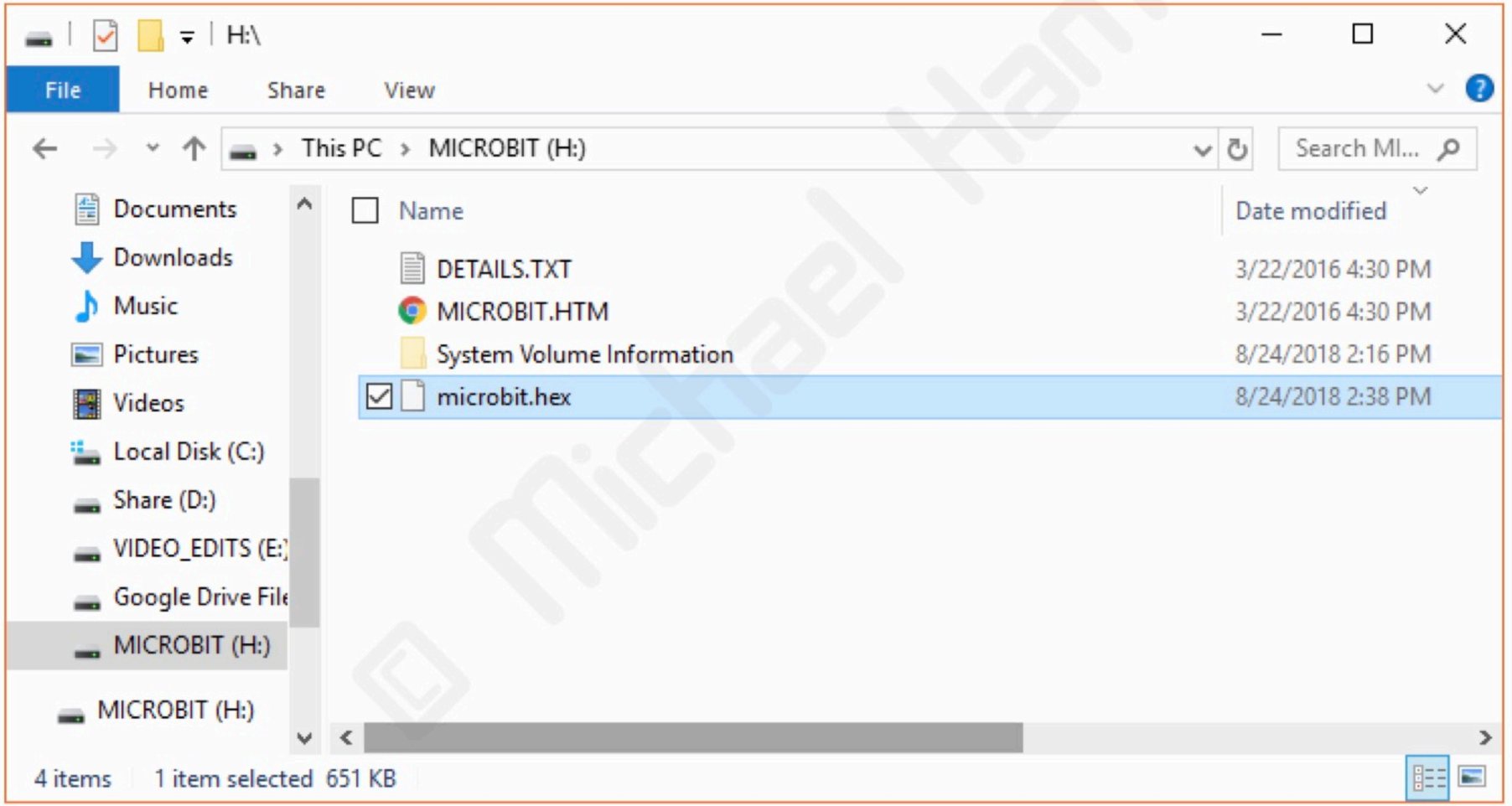



Blocks program to show a heart on the micro:bit

```
basic.show_icon(IconNames.HEART)

def on_forever():
    pass
basic.forever(on_forever)
```

# File Explorer



# 4. Build: Secret Channel Messenger

# Secret Messenger - Concepts

- Radio groups - we'll use numbers between 0 and 255
  - Only devices in the same group number can hear each other
  - Like tuning to an FM radio station
- Group number is our **pre-shared key (psk)**
  - If you know the group number, you can hear the broadcast
- Today's group number is: _____

# Secret Messenger - Live Code Python

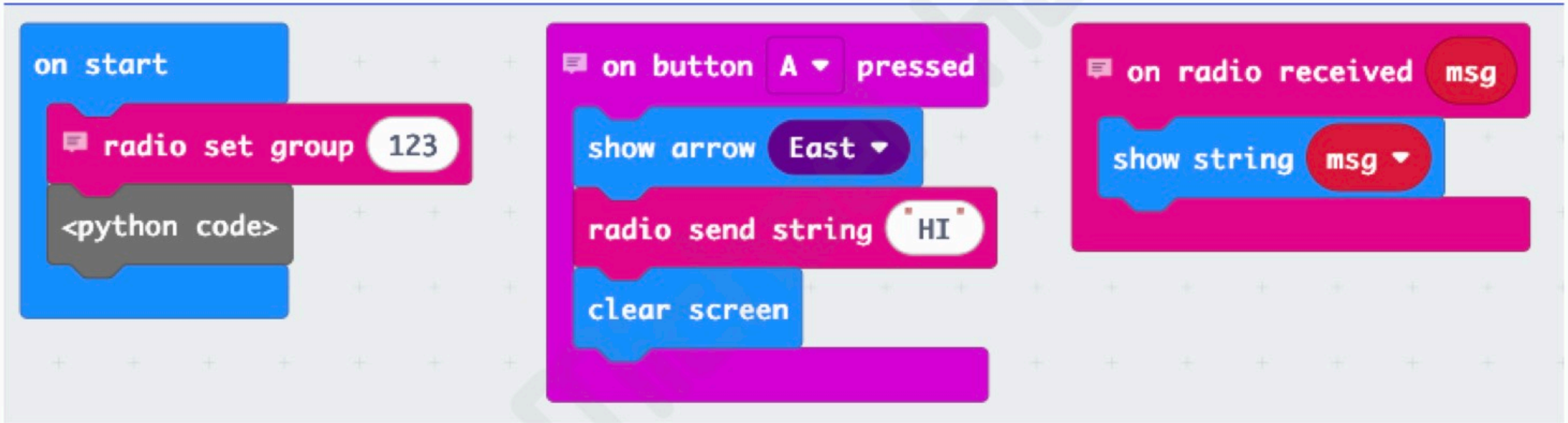
```
# send functionality, press A button to broadcast
def on_button_pressed_a():
    basic.show_arrow(ArrowNames.EAST)
    radio.send_string("HI")
    basic.clear_screen()
input.on_button_pressed(Button.A, on_button_pressed_a)

# receive functionality, show any incoming message
def on_received_string(msg):
    basic.show_string("" + (msg))
radio.on_received_string(on_received_string)

# set up the radio with group num and turn it on
radio.set_group(123)
radio.on()
```

Michael Ham

# Secret Messenger - Live Code Blocks




Blocks equivalent of the Python code to transmit messages on group 123

# Secret Messenger - Flash & Test

1. Download the **.hex** file and copy it to the **MICROBIT** drive
2. Pair up into groups of two.
    1. Person A presses button **A**. Person B's micro:bit should scroll "**HI**"
    2. Swap - Person B sends to Person A
3. Confirm both directions work
    1. If it doesn't work, check that both devices have the same code and the same group number

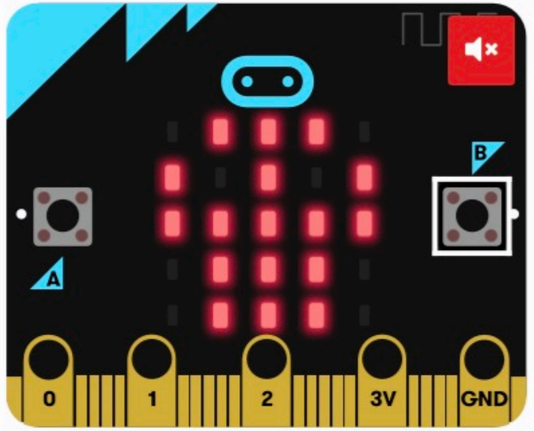
© Michael Ham

# If I change my group number to 0 (the default for micro:bit)…

**What happens to my communication?**
**If an attacker doesn't know our group number, can they read our messages?**
**Is this real security?**
**If not, what would actually make this secure?**

# Secret Messenger - Extension Challenge

- Add a panic button - **Button B** sends an alert
  - Send the string “ALERT”
  - Show a skull on the sender’s display
- If an alert is received:
  - Display a skull on the receiver’s display
  - Play the **SAD** sound efect




micro:bit displaying a skull

© Michael Ham

# Extension Challenge Python

```
# send functionality, press A button to broadcast
def on_button_pressed_a():
    basic.show_arrow(ArrowNames.EAST)
    radio.send_string("HI")
    basic.clear_screen()
input.on_button_pressed(Button.A, on_button_pressed_a)

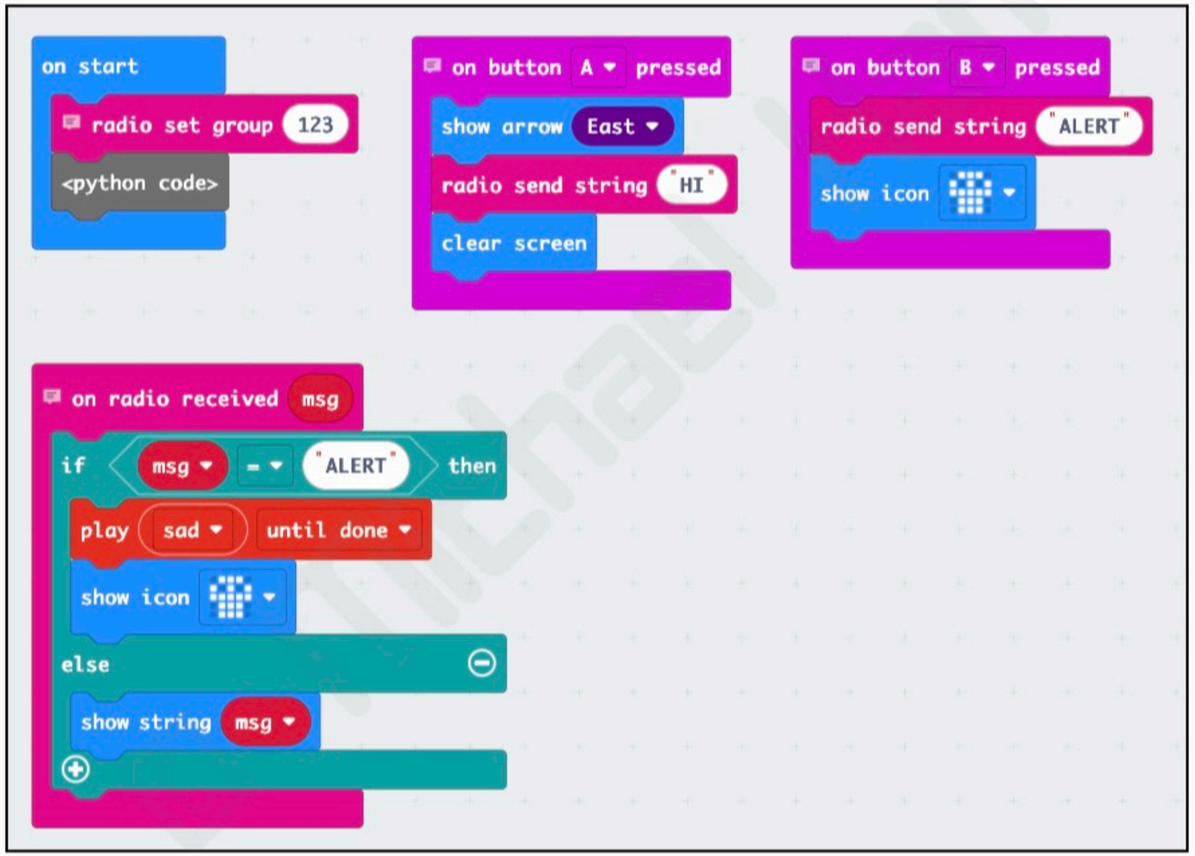
# send functionality, press B to send an alert
def on_button_pressed_b():
    radio.send_string("ALERT")
    basic.show_icon(IconNames.SKULL)
input.on_button_pressed(Button.B, on_button_pressed_b)

# receive functionality, show any incoming message
def on_received_string(msg):
    if msg == "ALERT":
        music.play(music.builtin_playable_sound_effect(soundExpression.sad), music.PlaybackMode.UNTIL_DONE)
        basic.show_icon(IconNames.SKULL)
    else:
        basic.show_string(msg)
radio.on_received_string(on_received_string)

# set up the radio with group num and turn it on
radio.set_group(123)
radio.on()
```

© Michael Ham

# Extension Challenge Blocks




Blocks version of the ALERT program

# 5. Wrap-up

© Michael Ham

# Anyone want to demo their extension code?

**No pressure, but if you got it working, it'd be cool to see!**

Michael Ham

# How would a real attacker find our group number if they didn’t know it?

**Think like an adversary! It doesn’t have to be fancy; it just has to work once.**

# Secret Messenger: Key Concepts

Michael Ham

# Secret Messenger: Key Concepts

- **Pre-shared key (PSK)** - a secret value both sides agree on before communicating
  - Our group numbers are PSKs

Michael Ham

# Secret Messenger: Key Concepts

- **Pre-shared key (PSK)** - a secret value both sides agree on before communicating
  - Our group numbers are PSKs
- **Radio groups/channel** - a number (0-255) that filters what messages a micro:bit pays attention to

© Michael Hall

# Secret Messenger: Key Concepts

- **Pre-shared key (PSK)** - a secret value both sides agree on before communicating
  - Our group numbers are PSKs
- **Radio groups/channel** - a number (0-255) that filters what messages a micro:bit pays attention to
- **Security by obscurity** - hiding how a system works as the only defense

© Michael Hall

# Secret Messenger: Key Concepts

- **Pre-shared key (PSK)** - a secret value both sides agree on before communicating
  - Our group numbers are PSKs
- **Radio groups/channel** - a number (0-255) that filters what messages a micro:bit pays attention to
- **Security by obscurity** - hiding how a system works as the only defense
- **Encryption** - scrambling the message so someone with the right channel can’t read it without the decryption key

# Secret Messenger: Key Concepts

- **Pre-shared key (PSK)** - a secret value both sides agree on before communicating
  - Our group numbers are PSKs
- **Radio groups/channel** - a number (0-255) that filters what messages a micro:bit pays attention to
- **Security by obscurity** - hiding how a system works as the only defense
- **Encryption** - scrambling the message so someone with the right channel can't read it without the decryption key
- **Eavesdropping** - passively listening to communications you weren't meant to receive

# Real World Connections

- Tie to real security systems
  - Group number maps to pre-shared keys in WPA2, Bluetooth pairing pins, symmetric encryption
- Honest limitations
  - Channel isolation is not encryption
  - Real-world security layers on encryption to channel selection (HTTPS over Wi-Fi)

Michael Ham

# Please bring your micro:bits and USB cables up to the front.

**Thanks for joining!**

# Bonus: Scanner

# Channel Group Scanner Python

```
current_group = 0

# set up radio to show any intercepted messages & group number
def on_received(msg):
    music.play(music.builtin_playable_sound_effect(soundExpression.spring),
music.PlaybackMode.UNTIL_DONE)
    basic.show_string(str(current_group) + ":" + msg)
radio.on_received_string(on_received)
radio.on()

# scan through all channels (0-255) with a 200ms delay
def scan():
    global current_group
    while True:
        radio.set_group(current_group)
        basic.pause(200)
        current_group = (current_group + 1) % 256

control.in_background(scan)
```